Certify-or-Refuse: A Machine-Checked Soundness Boundary for Untrusted Agent Edits to Opaque-Semantics Artifacts

2026-07-10

Abstract

An LLM agent that edits a spreadsheet, a database schema, or a dbt project produces an artifact that *opens fine* but may be silently wrong: a structural edit that fails to propagate references corrupts computed values with no visible symptom, and neither the agent nor a human reviewer can see it without the defining engine. We ask which such edits can be certified **engine-free** — offline, against the artifact's own structure, trusting neither the editor nor a recomputation — and we answer by machine-checking **both sides of the boundary** (Lean 4, no `sorry`, axioms `propext` and `Quot.sound` only), with the remaining middle ground stated open. On the certifiable side: the premise of our invariance theorem — the edit's reference-dependency graph is a function-preserving relabeling of the original's — is *decidable from syntax*, and we ship an executable decision procedure `check` with a machine-checked soundness theorem: if `check` accepts, every computed value transports across the edit under **every possible engine** (the engine is universally quantified, never run), carrying the artifact's embedded cached values as a self-oracle. On the uncertifiable side: an impossibility theorem — an edit introducing function semantics unwitnessed in the original admits two engines that are pointwise indistinguishable through the original artifact yet disagree on the edited value, so *no* engine-free checker can certify it. Certify-or- refuse is therefore not a design choice but the only sound shape, and our checker's accept class is exactly the *proven-certifiable* relabeling class — every accept is certifiable; whether the certifiable class extends further (edits reusing witnessed semantics) is stated open. A companion locality theorem factors mixed edits into a certified structural scaffold plus a bounded value-fill audit cone. The theory is load-bearing in the running system: the deployed checker agrees with the Lean decision procedure on a randomized differential battery, a production certifier (`xlq certify`) covers five structural operations on real spreadsheets, and the same format-parametric core certifies dbt model refactors engine-free — catching dangling references and silent logic changes that re-materialization confirms, while certifying faithful renames with no SQL executed. We are deliberate about what is *proof* and what is *corroboration*: the machine-checked boundary is the result; the empirical harness (147/147 corrupted foreign edits refused; a naive edit path silently corrupts 85.5% of development-tier workbooks — confirmed-genuine after our own study corrected two

mislabels — while the certified path corrupts none; and a pre-registered, run-once **locked test on corpora development never touched** — EUSES, Enron, and a production dbt project — in which the guard made **zero false certifications across 518 foreign edits (503 refused, 15 fail-closed errors)**, the shift arithmetic made zero errors in 283,960 real cells, measured fail-closed cost of 19.6–32.0%, and the one real defect found sat exactly in the layer our theory declares trusted-not-proven, since fixed) corroborates it, and we report each measurement with its confound — including two false certifications our own adversarial reviews found in earlier versions of the system, which we closed and report as fixed defects.

1. Introduction

The scarce resource in the agentic era is not capability but **verifiability**. A capable model will, with high probability, perform a structural spreadsheet edit correctly; but "high probability" is exactly the wrong guarantee for a financial model, a payroll sheet, or a regulatory filing, where a single unpropagated reference silently changes a computed total and ships. The failure is invisible by construction: the edited file is a valid `.xlsx`, it opens, and the wrong numbers sit in cells whose formulas *look* plausible. You cannot see the corruption without the engine, and asking the engine is asking the very component whose edit you are trying to check.

This paper asks a narrower, more durable question than "can the model do the edit?": **can we certify that a given edit is value-faithful, offline, against the artifact's own structure, without trusting the editor and without running the defining engine?** The answer we prove is yes for the *structural* fragment — the coordinate-relabeling edits (insert/delete rows and columns) that are simultaneously the most common agent operations and the ones whose damage is most invisible. The certifier accepts a structural edit only when its reference-dependency graph is the relabeling of the original's, and refuses otherwise. Because the guarantee is grounded in the artifact rather than the model, it does not decay as models improve — it is the boundary a capable agent should be *required* to pass, not a crutch for a weak one.

Our claims, in order of strength:

1. **(The boundary, both sides machine-checked.)** We *bracket* engine-free certification of agent edits: function-preserving relabelings of the reference-dependency graph are certifiable — witnessed by an executable decision procedure `check` for that premise with a machine-checked soundness theorem quantifying over **all** engines — while edits introducing function semantics unwitnessed in the original are uncertifiable by *any* engine-free checker (impossibility: indistinguishable engines disagree), so *certify-or-refuse is the only sound shape* (§3, Lean 4, no `sorry`). The middle ground (edits reusing witnessed semantics at new positions) is stated open.
2. **(Composition.)** A locality theorem factors a mixed edit into a certified structural scaffold plus value fills whose effect is provably contained in their downstream cone — collapsing the audit surface from the whole artifact to a bounded set (§3, measured in §5).
3. **(Mechanism, theory-linked.)** A certify-or-refuse router whose deployed

checker implements `check` and agrees with the Lean decision procedure on a randomized differential battery; a production certifier (`xlq certify`, five structural ops) in translation-validation mode; a hardened, engine-corroborated trusted parse; and fail-closed boundaries for everything outside the verified surface (§4).

4. **(Generality + corroboration, reported with confounds.)** The same format-parametric core certifies dbt model refactors engine-free (§5.6); an engine-free foreign-edit battery, an independent-oracle A/B, and a diverse-corruptor confusion matrix support the theory on real artifacts, each reported with its limitation, not as independent proof (§5).

A recurring, honest theme: three rounds of adversarial review of our own artifacts found real defects — silent-corruption bugs in the trusted tokenizer, a circular experiment, and an unimplemented soundness boundary — each of which we fixed and report as a fixed defect, because the discipline of finding them is part of the contribution (§6).

2. The problem: opaque-semantics artifacts and invisible structural damage

A spreadsheet is a program whose semantics live in an engine (Excel, LibreOffice Calc, IronCalc) that most tools do not reimplement. An edit tool that manipulates the file's bytes therefore edits a program it cannot evaluate. The dominant failure mode is the **unpropagated reference**: insert a row above a formula's input and, unless every reference below the insertion point is shifted, the formula now reads the wrong cell. The popular `openpyxl` library's `insert_rows` does exactly this — it moves cell contents but rewrites zero references — so on any workbook with a below-insertion reference, it produces a file that opens cleanly and computes wrong (§5.2).

The artifact carries its own partial ground truth: Excel writes a cached value `<v>` next to each formula. This *self-oracle* is what a certifier can check against — but only for cells the artifact chose to cache, and only if the checker can recompute, which returns us to the engine. Our theorem removes the recompute: for the structural fragment, value-faithfulness follows from structure alone.

3. The formal core (the contribution)

We model a computation as `C = (fn, deps)`: `fn n` maps a node's ordered dependency *values* to its value, and `deps n` is its ordered dependency *nodes*. This is exactly what a formula carries — what it reads and how it combines what it reads — with `fn` left an arbitrary (opaque) function. Evaluation is fuel-bounded: `eval C 0 _ = default` and `eval C (k+1) n = fn n (map (eval C k) (deps n))`. The theorem holds at every fuel level, hence for the true evaluation of any acyclic computation.

Theorem 1 (exact structural certification). Let σ map every node of `C` to a node of `C'` with the same function and dependency list relabeled by σ (a function- and edge-preserving graph isomorphism): `C'.fn (σ n) = C.fn n` and `C'.deps (σ n) = map σ`

`(C.deps n)`. Then `eval C' k (σ n) = eval C k n` for all `k, n`.

Consequence. If the original's evaluation is the artifact's embedded ground truth `O` (`eval C k = O`, the self-oracle), then a structurally-faithful edit reproduces that ground truth at the relabeled positions, `eval C' k (σ n) = O n`, established without the defining engine and without recomputing `O`. This is the exact tier of certify-or-refuse.

Theorem 2 (locality / audit-surface bound). Define `agree_upto C C' k n`: the two computations have identical `fn` and `deps` at every node within `k` dependency-steps of `n`. Then `eval C k n = eval C' k n` — a node's value depends only on its dependency cone. Hence a value edit changes nothing outside its downstream cone: a mixed edit factors into a certified structural scaffold plus value fills whose effect is provably contained, collapsing the audit surface from "did this corrupt anything anywhere?" to "are these N cells right?".

Both theorems are machine-checked in Lean 4, self-contained (no Mathlib), with `#print axioms` reporting only `[propext, Quot.sound]` and no `sorry` (`formal/SelfOracle.lean`). The reference-shift algebra's arithmetic laws (insert∘delete = identity, monotonicity, the six-case delete clamp against the set-theoretic truth) are separately proved for all inputs by the Z3 SMT solver (`formal/shift_laws.py`).

Theorem 3 (graph preservation — the shift discharges the hypothesis, for single cells and range endpoints). Theorems 1 and 2 *assume* the edited graph is the σ-relabeling of the original's; the tool does not assume it, it produces it by shifting references. We model a formula as a list of tokens — each an opaque literal (number, string, function name), a single cell reference, or a range carrying its two endpoints — and machine-check that the reference-shift produces exactly the relabeled graph: `refs (shiftF σ f) = (refs f).map σ`, with literals provably untouched (`formal/RefShift.lean`). This is *verbatim* the `hdeps` premise of Theorem 1, so for these token shapes the graph isomorphism is discharged **constructively by the tool's operation**, not assumed. We also machine-check `delete∘insert = id` on the cell maps. All `sorry`-free, axioms `[propext, Quot.sound]`.

What Theorem 3 does and does not give. It is a fusion law parametric in an *arbitrary total* map `σ : Cell → Cell`: it proves the shift *transports* the reference graph by whatever σ it is given — it does **not** verify that σ is Excel's shift arithmetic (that is the separate Z3-proved algebra), nor does the total-map model express the asymmetric six-case delete *clamp* (head-in-band → k, tail-in-band → k−1), the `#REF!` a fully-consumed reference produces, or the true multi-cell interior of a range (we model a range by its endpoints). So the honest statement is: **the token→graph→value chain is machine-checked for single-cell references and range-endpoint pairs under a value-preserving relabeling**; the explicitly *trusted* surface is the byte→token parse (§4, §5.1), the correctness of σ itself, the asymmetric clamp / `#REF!` algebra, range-interior dependencies, and sheet-qualified/3D/table/external references. This is a real narrowing of the trusted base — not its elimination. Fidelity of value edits remains out of the exact tier by construction.

Theorem 4 (the premise is decidable, and the decider is proven sound — `formal/Checker.lean`). Theorems 1–3 would still be decorative if the running system never checked their hypothesis. We therefore state the hypothesis of Theorem 1 as an **executable decision procedure** over syntax alone: `check` S_0 S_1 σ verifies that ev-

ery checked node's opaque function *skeleton* is preserved (`S₁.skel (σ n) = S₀.skel n`), that its dependency list is the σ-image (`S₁.deps (σ n) = map σ (S₀.deps n)`), and that the checked domain is closed under dependencies. The soundness theorem, `check_sound`, then proves: if `check` returns `true`, every checked node's value transports across the edit **for every interpretation `I` of the skeletons** — the engine is universally quantified and never run — and `check_transports_oracle` carries the artifact's embedded cached values to the edited positions. The procedure *executes* (`#eval` in the development runs it: a faithful relabeling returns `true`; an argument-order botch and an operator botch return `false`). The consequence for the system is architectural: a certifier implementing `check` accepts **any producer's faithful edit** — regardless of byte-level differences, cache handling, or which tool made it — and its soundness is this theorem, not equality to a reference transform.

Theorem 5 (impossibility — the other side of the boundary, `formal/Impossibility.lean`). An edit that introduces a function skeleton *not witnessed anywhere in the original artifact* cannot be value-certified by any engine-free checker. The proof is indistinguishability: overriding an engine at an unwitnessed skeleton changes no evaluation of the original artifact (`eval_override_fresh` — the two worlds agree at every node and every fuel, hence realize every cached observation identically), yet the edited node's value can be made to differ from *any* value a checker might commit to (`fresh_skeleton_uncertifiable`), and two such worlds disagree with each other (`two_worlds_disagree`) — and the quantification over checkers is itself a machine-checked object: `no_engine_free_predictor` models a checker's value commitment as *any* function of the two syntactic artifacts and proves every such predictor is refuted by an engine consistent with the original. Sound checkers **must refuse**. One scope note, stated plainly: the impossibility is for *semantics-agnostic* checkers — a checker carrying a trusted specification of particular function semantics (say, documented Excel built-ins) has a partial engine and escapes the hypothesis; that is an oracle assumption, exactly what "engine-free" excludes. Within that scope, certify-or-refuse is the only sound shape — proven, not a design preference.

The boundary, stated. Together, Theorems 4 and 5 bracket engine-free certification: edits whose reference graph is a skeleton-preserving relabeling are certifiable, with an executable proven-sound decider (Theorem 4); edits introducing unwitnessed semantics are uncertifiable by any engine-free checker (Theorem 5). The honest middle ground — edits that *reuse* witnessed skeletons at new positions, such as copy-pasting an existing formula shape onto new inputs — is not settled by either theorem and is stated as open: its values are forced by the original's syntax only when the reused skeleton's dependencies carry observed values, a case our current checker routes to refusal.

4. The certify-or-refuse router and its trusted base

The router. Given an original artifact and an untrusted foreign edit plus a declared structural op, the router computes the op's node relabeling σ, and CERTIFIES iff the foreign edit's reference-dependency graph is exactly the σ-image of the original's; otherwise it REFUSES. Two properties make this sound as a check of *untrusted* work: under-declaring a change is caught (an unaccounted graph difference → refuse), and

a wrong σ yields a mismatch → refuse. The router is fail-closed: any condition it cannot certify becomes a refusal.

Two certifier modes, honestly labeled. The system ships two implementations of the router, and we are precise about which carries which guarantee. *(i) Direct-premise mode* implements Theorem 4's `check` literally: extract both artifacts into `(skeleton, deps, oracle)` triples and decide the premise — skeletons preserved, dependency lists the σ-image, containment (no unaccounted nodes), oracle values preserved outside any declared fill cone. Its accept class is exactly the certifiable class of §3 — any tool's faithful edit, regardless of bytes — and its soundness is `check_sound`. The deployed implementation agrees with the Lean decision procedure (run via `#eval`) on a randomized differential battery of faithful edits and four botch classes — skeleton change, dependency reorder, retarget, dropped dependency — at 30/30 (`formal/differential_check.py`); we state plainly that this differential link is evidence of implementation fidelity, not a proof of it. *(ii) Translation-validation mode* is the production Rust path (`xlq certify`): apply the tool's own proven structural transform to the original and diff the foreign edit against it, certifying iff the only differences are stripped caches and number formats. It is stricter than mode (i) (byte-for-formula identity to the canonical transform), engine-free, and multi-sheet-safe; its marginal value over *doing* the edit is trust-topology — the untrusted producer's artifact ships, gated fail-closed, with the tool off the write path — not a verifier-cheaper-than-prover asymmetry (there is none in this mode; it recomputes the transform). Mode (i) is where the theory is load-bearing; mode (ii) is the hardened production gate.

The trusted base: the reference-shift tokenizer. Every certificate is only as sound as the predicate deciding *which tokens in a formula are cell references*. We initially used syntactic proxies (a 1–3-letter column, an unbounded row) and adversarial review found these were wrong in three ways that caused *silent corruption*: a function name whose prefix looks like a cell (`BIN2DEC` → `BIN3DEC` under a row insert), a function name ending in digits before a paren (`LOG10` → `LOG11`), and out-of-grid tokens (`XFE9`, `A2000000`) whose column or row exceeds the sheet limits. We replaced the proxies with the exact **grid-validity** predicate — a token is a cell reference iff its column is in `A..XFD` (1..16384) and its row is in `1..1048576`, boundary-gated so an identifier or function call is never mistaken for a reference. This one rule subsumes all four bug classes.

We validate the tokenizer at two levels. First, **differential cross-checking** against an independent A1 shifter over 19 digit-bearing Excel functions, out-of-grid tokens, `$`-absolutes, and ranges, across insert-rows and delete-rows: 175 (formula, op) pairs, 0 disagreements (`tokenizer_fuzz.py`). This establishes *mechanization consistency* between the Rust scanner and the reference — but the reference is a second implementation of *our own* grid-validity spec, so it cannot establish conformance to a spreadsheet engine's semantics. Second, therefore, **differential value-preservation against an independent engine**: a blank-row insert is value-preserving under a correct reference shift, so recomputing the same file with LibreOffice *before and after* the edit must leave every formula's value unchanged (`tokenizer_conformance.py`, seeded property-based generation over evaluable formulas — digit-bearing function names, single/mixed/absolute refs, ranges, strictly-

distinct cell values so a mis-shift onto another cell changes the value). We run this across **all four structural ops** (insert/delete × rows/columns) and against **two independent engines** — LibreOffice *and* the pure-Python `formulas` engine, neither of which is xlq's IronCalc (`conformance_v2.py`): **465 formulas checked per engine, 0 value divergences, with the two engines agreeing exactly.** (The earlier single-op/one-engine run, `tokenizer_conformance.py`, gave 264 formulas / 0 divergences.) We are precise about what this does and does not establish. Value-preservation is *necessary but not sufficient* for reference-graph correctness: a mis-shift of a reference that the formula's value does not depend on (a zero-weighted term, a non-max argument of `MAX`) would pass undetected, so the count over-counts references actually *witnessed* by the value. Two independent engines agreeing rules out a single-engine artifact, but both are open-source engines, not Excel; and sheet-qualified/3D/table/R1C1 constructs are outside the generator. The honest label is *differential value-preservation against LibreOffice — a necessary-not-sufficient corroboration of the trusted parse, complementing the same-spec fuzzer* — not full conformance to an engine's reference semantics.

The one undecidable case, made a fail-closed boundary. A defined name spelled like a grid-valid cell (`FY2021` = column FY, row 2021; `Q1`; `Tax2020`) is indistinguishable from a reference *in the formula text*, so the tokenizer would shift its uses and the edited file would still equal the tool's own (wrong) transform — the single place `certified ⇒ correct` could be false on a real workbook. It is, however, decidable from the workbook's defined-names table, which the certifier already holds. We close it fail-closed: a structural edit on a workbook containing a defined name that collides with a cell-reference spelling emits a `defined_name_ref_collision` residual, and the certifier refuses. A workbook defining `FY2021` and using it in a formula routes to REFUSE; a benign name (`TaxRate`) proceeds. This converts the last hidden hole into a demonstrated boundary and scopes the verified surface explicitly: **single sheet, in-grid coordinates, row/column shifts, no name collision.**

5. Corroboration on the development-tier corpus (reported with confounds)

Sections 5.1–5.8 measure on the vendored fixture corpus — real-provenance files, but our de-facto *development tier* (the locked test tier is §5.9).

None of the following is the soundness argument — that is §3. Each supports the theorem on real files and each carries a limitation we state.

5.1 Engine-free certification of untrusted foreign edits

Running the router (format-parametric core) on `openpyxl`'s edits of 172 real formula-bearing workbooks, engine-free, cross-checked against an independent LibreOffice oracle: **0 false certifications and 147/147 corrupted foreign edits refused** (`foreign_certify`). The load-bearing caveat, always reported alongside: this soundness is bought with heavy refusal — the fail-closed A1 proxy certifies only about **1 in 23** *faithful* foreign edits, because it refuses every reference form it cannot fully model (whole-row, whole-column, cross-sheet, table, defined-name). Useful

soundness — a high certify rate on faithful edits — requires the complete parser of §4, not the proxy. The value is the boundary, not the throughput.

5.2 Independent-oracle A/B: the corruption the boundary prevents

On 172 development-tier workbooks, insert-row@2, with LibreOffice (independent of both `openpyxl` and the tool's engine) recomputing each edit against Excel's cache: the naive `openpyxl` edit path silently corrupts 149/172 workbooks *as originally measured* — of which **147/172 = 85.5% is confirmed-genuine reference corruption** after our own coincidence-bound study (§5.8) surfaced two label mislabels (files whose formulas carry *zero references of any kind*, where the flagged divergence is LibreOffice recomputing `ACCRINT` differently from Excel's cache, not corruption; three further files carry only non-A1 reference classes where corruption is plausible but not attributable by this oracle). The tool's structural edit is engine-confirmed faithful on 150/172 and explicitly refuses the remaining 22 — **0 silent corruptions** (`agent_ab`). We extend this across **all four structural ops and both independent engines** on generated distinct-value workbooks (`agent_ab_v2.py`): the naive path silently corrupts insert-rows 100% / delete-rows 98.8% / insert-cols 94.3% / delete-cols 80.2% (**93.8% overall**), the tool **0% on every op**, and the `formulas` engine agrees with LibreOffice *exactly* — so the protection is not a LibreOffice artifact and holds across the op space, not just insert-row@2. We flag the confound that remains: the openpyxl figure is a corpus/tool property (openpyxl shifts no references), not an agent-error distribution, and a competent reference-shifting engine also gets these ops right; the tool's differentiator is auditability and explicit refusal, which the A/B does not isolate. We found and fixed an oracle false-positive during this study — `ROW()/COLUMN()` are position-dependent and legitimately change on a row insert — which is why the oracle excludes position-dependent functions.

On the **real** corpus across all four ops we check shift correctness *deterministically* rather than by recompute, because real-corpus value-preservation is confounded — LibreOffice recomputes exotic financial/date functions (ACCRINT, CUMPRINC, DB, DAYS360, TIME) inconsistently with the Excel cache, flagging as "corrupt" files whose shifted formulas we verified correct by hand. So we compare xlq's output formulas to the two-engine-validated reference shifter over ~6,000 real formula cells (`shift_correctness_real.py`): **xlq is 100% correct on every op** (insert-rows 1651 cells, delete-rows 1608, insert-cols 1648, delete-cols 1076; 0 mismatches), while the naive path leaves 17–72% of shift-requiring cells wrong. Constructs the simple checker cannot independently verify (whole-column/row, cross-sheet, tables, range-with-function endpoint) are skipped, not guessed; every early "mismatch" was a checker bug (comparing a deleted-band cell to the wrong output cell; the reference shifter not handling `F:F`, which xlq correctly shifts to `G:G`), and xlq had zero real errors.

5.2b A fifth op — move-rows — shows σ is not special to insert/delete

Theorem 1 holds for *any* function-and-dependency-preserving isomorphism σ, so the certifiable class is not limited to the monotonic insert/delete shift. We add **move-rows** (relocate a contiguous block of rows), whose σ is a *permutation* — proved a

bijection in Z3 (injective + surjective on `[1, maxrow]`, both directions) — and whose physical edit reorders rows (a buffered, non-streaming rewrite). A range whose endpoints reorder under the permutation cannot be a shifted rectangle, so it fail-closes as a `move_straddles_range` residual. On the same real corpus, deterministic move-rows shift correctness is **100% on 1,538 real formula cells** (25/60 workbooks fail-closed as residual/straddle), and the certify-or-refuse contract holds: `certify(orig, correct move)` CERTIFIES while a botched move (one reference reverted) is REFUSED. This is a concrete demonstration that the exact tier extends to a genuinely non-monotonic structural edit by supplying the right σ, with no change to the theorem.

5.3 Independent-oracle confusion matrix for the production certifier

We adjudicate `xlq certify`'s verdicts *independently of the tool* over **diverse** corruption. An initial matrix used a single corruptor — `openpyxl`'s deterministic no-op-shift — and the Excel cache with a reliability gate; its `FN=0` on 14 edits was a point estimate (rule-of-three ≈ 21%) confounded with editor identity. We diversified to three corruptor types — `openpyxl` (no-op), `unshift_one` (revert one shifted reference), and `wrong_delta` (over-shift one reference) — with ground truth **by construction** (we inject the corruption, so its label is independent of any engine) and a LibreOffice *self-consistent* oracle as cross-check (recompute before/after; no reliability gate). All 45 injected corruptions were refused (each type 15/15) and all 15 faithful tool-produced edits certified (`cert_confusion_v2`). We are careful about what this does *not* show: because the certifier accepts iff the edit equals the tool's transform, and all three corruptors are *defined* as deviations from that transform, `FN=0` here is **by construction**, not a sampled bound — we do not attach a confidence interval to a structurally-guaranteed zero, and `unshift_one`/`wrong_delta` differ only in the sign of a perturbation the certifier does not weigh. The one *reachable* false certification is not in this family at all: the cell diff compares only sheet cells, so a foreign edit that shifts every cell formula correctly but leaves a **non-cell reference** (a defined name's target, a data-validation or conditional-formatting formula, a chart series) unshifted is invisible to it. We built exactly that edit (`cert_noncell_test.py`): with a defined name `Rate = $A$10` left unshifted while all cells moved, an earlier `xlq certify` **CERTIFIED it with all-zero diffs — a genuine false certification.** We closed it: certify now checks that defined names match the tool's transform (refusing the unshifted one, certifying the correct one) and *fail-closes* on data-validation, conditional-formatting, and chart parts it does not compare — a stated coverage boundary. Separately, 6 of the cell-formula corruptions were **value-preserving** (a reference error that does not change the current value); the value oracle called them faithful but certify refused all 6 — its structural equality is *stricter* than a value check. The symmetric cost, stated plainly: certify equally refuses a *faithful but non-identical* rewrite (a commuted `A1+B1` → `B1+A1`), so its accept class is "byte-for-formula identical to the tool's transform," narrower than "value-faithful."

5.4 The interventional finding: differential testing hardened the trusted base

We ran a live fast-model agent on 20 real workbooks, tasking it to rewrite formula references for insert-row@2, and compared its output to the tool's transform. As a guard-soundness experiment this is **circular** — the "agent correct" label and the `xlq certify` verdict are the same predicate applied to a file grafted onto the tool's own output — and we do not report it as such. Its genuine, non-circular yield is **differential testing that surfaced real silent-corruption bugs in the tool's own tokenizer** (`BIN2DEC`, `Sales2020`, and on follow-up `LOG10` and the out-of-grid class), each now fixed and covered by §4's cross-check. A validation method that finds real bugs in its own trusted base is exactly what a certifier's TCB needs; the honest framing is that the method proved its worth, not that it produced a soundness number.

5.5 Composition on mixed edits, measured

By Theorem 2 a mixed task factors into a certified structural scaffold plus value fills contained in their downstream cone. We measure this on realistic mixed edits (`composition_coverage.py`): every scaffold is certified, and the audit surface collapses to the fill cone — 100/84/68/52% of the artifact certified untouched as the number of fills grows (mean 76%) — so on a realistic edit-distribution study the certifiable *component* rises from 27% of tasks fully certified to **87% with a certified scaffold**.

5.6 A second real domain: dbt model refactors, certified engine-free

The theory is format-parametric — nothing in §3 mentions spreadsheets — and we cash that on a practitioner-relevant second domain: **dbt projects**, where models reference each other via `{{ ref(…) }}`, the everyday edit is a rename/refactor, and the warehouse's materialized tables from the pre-edit state are the self-oracle. The adapter extracts the same `(skeleton, deps, oracle)` triple the checker consumes: skeletons are SQL with references abstracted to ordered slots (case and whitespace folded *outside* string literals — `'ABC'` is never conflated with `'abc'`; any unmodeled Jinja marks the node dynamic and out of the exact tier, refused rather than guessed); the edited artifact is built with **no SQL executed**. Four scenarios on a ten-node staging→marts project: a faithful rename with all references updated is **CERTIFIED** (100% of the artifact untouched, no engine run); a rename leaving one downstream reference dangling is **REFUSED**; a rename with a silently changed aggregation (`SUM→AVG`) is **REFUSED**, and independent re-materialization confirms the values really differ; a rename plus a *declared* logic change is certified as a scaffold with the audit surface exactly the declared model plus its downstream cone (collapse 0.8), and re-materialization confirms outside-cone values are preserved. Honest scope: model-level granularity (the cone is whole models, not columns); a mini-dbt subset (single-argument `ref`/`source`; no macros); and fail-closed normalization — a comment-only reformat is refused, never wrongly certified.

5.7 The interventional study: live agents, guarded vs unguarded, with cost

Finally, the experiment every earlier version of this work lacked: **live agents' own errors**, caught by the direct-premise checker, with completion cost measured. Design, with the independence structure stated: 21 real-workbook tasks (196 formula cells; task prep excludes constructs the guard cannot see, so measured cost *understates* real-world refusal cost — stated, not hidden); the agent's artifact is built via `openpyxl` plus surgical formula splicing with the reference tool nowhere in it; ground truth is the two-engine-validated reference shifter (cells outside its grammar are excluded and counted, never guessed); the guard is the direct-premise checker of §4.i; and two live agent conditions (a careful and a hasty prompt, same fast model) make the error distribution the agent's own rather than injected. The harness was first validated on synthetic perfect/sloppy agents (perfect: zero corruption in both arms; sloppy: all seven injected corruptions blocked).

Results: the careful agent erred on 2/21 tasks, the hasty agent on 4/21 (six saves total across the two arms — which share tasks and model, so they are not independent runs — spanning four distinct workbooks; with n this small we report counts, not rates: 6/6 blocked has a 95% binomial lower bound of 0.54). Unguarded, all six erroneous artifacts ship as silent corruption; four are content errors and two are protocol-keying errors (the agent's formulas were right but returned under shifted addresses — interface-dependent corruption, still corruption as shipped). Guarded, **zero ship — all six are refused, with zero false certifications on the 137/196 truth-visible cells and zero unambiguous completion cost**. The symmetric caveats, both stated: certified artifacts contain truth-blind cells (11 of 19 certified tasks carry cells outside the truth grammar), where corruption would be invisible to the truth instrument just as hidden saves are — the instrument cannot contradict the guard there in either direction; and the two refusals of "correct" hasty work are on truth-partial tasks whose unverifiable cells may themselves be wrong — possible hidden saves, split-reported. Two findings deserve emphasis. First, the live error distribution *naturally* contained the adversarial classes our earlier synthetic corruptors were criticized for omitting: partial shifts (`F.INV(F2,B3,C3)` — two references shifted, one missed), a protocol misread (formulas returned under already-shifted addresses), and dropped cells. Second, one careful-agent error was a **semantics-changing edit with correctly shifted references** — `TEXTSPLIT(A26,,"")` rewritten as `TEXTSPLIT(A27,"")`, silently turning a row-delimiter argument into a column-delimiter — and the graph check caught it because the function *skeleton* changed even though every reference moved correctly: precisely the class a value-spot-check or a reference-only linter would miss.

5.8 The probabilistic tier, quantified: the coincidence bound

Everything outside the certifiable class routes to probabilistic checking against the self-oracle — and until now "probabilistic" was unquantified. We derive the **coincidence bound**: the probability that a *wrong* edit passes a k-cell value check because the misread cells coincidentally carry colliding values. The naive independent bound `q^k` is badly optimistic: on the real corpus the honest, dependence-aware mixture

bound (within-file value repetition dominates) is **30×** higher at k=5 and **~2×10⁴×** at k=10, and tracks a Monte-Carlo simulation on real formulas within ~25% at every k. Measured on 230 real first sheets under Excel read semantics, the off-by-one-row collision rate — exactly the naive edit path's error — is $\hat{q}$ = 0.178 pooled (file median 0.125, p90 0.40; spreadsheets are full of repeated values). Detection therefore needs **k = 5 checked cells for 99% and k = 9–10 for 99.9%** (the mixture bound is itself still mildly optimistic within-file — the MC sits just above it at k = 10) against *systemic* errors — double the naive prescription. Against *localized* errors the tier is coverage-bound, not collision-bound: a single mis-routed reference corrupts one cell plus its cone, so a k-of-N sampled check detects it with probability at most k/N regardless of q. And the hard limit, engine-verified **on this corpus** (engine calc-test suites plus templates — not in-the-wild business workbooks): **of 161 workbooks where the naive path's error is genuinely present, 19 (11.8%; conservatively 12 = 7.5%) pass a full k = N value check** — LibreOffice recomputes identical values on a *wrong reference graph* (aggregates, `MIN`/`MAX`, `MODE`-class functions absorb the misread). The passing files are dominated by test suites for exotic non-injective functions, over-represented here by construction; the floor's magnitude plausibly *shrinks* on arithmetic-dominated business sheets, and we state that direction rather than claim transfer. This is the quantified case for the exact tier: value checking, even exhaustive, cannot close the gap that the graph check closes by construction.

In-the-wild transfer, measured (locked test, §5.9): on 761 EUSES and 777 Enron first sheets the off-by-one collision rate is *higher* than this corpus — pooled $\hat{q}$ = 0.478 and 0.566 respectively (file-median 0.104/0.245), pushing 99.9%-detection to k = 12 and **k = 20** checked cells. The direction we pre-stated held: the probabilistic tier is substantially *weaker* on real business sheets, which strengthens, not weakens, the case for the exact tier.

5.9 The locked in-the-wild test: pre-registered, run once

Every number above §5.9 is measured on the vendored fixture corpus — our de-facto development tier. To answer the ecological-validity question, we ran a **locked test**: protocol and nine predictions committed to the repository *before* any test data was downloaded (research-log/016); corpora untouched by development — **EUSES** (796 converted workbooks, CC-BY-4.0, checksums matched to the canonical Zenodo record) and **Enron** (786, CC0, FERC-released business spreadsheets), both `.xls→.xlsx` via LibreOffice (disclosed caveat: cached values are engine-regenerated). One sampling caveat the pre-registration did not anticipate: conversion capacity forced a deterministic *lexicographic-prefix* subset at acquisition (first 800 sorted paths per corpus; Enron additionally a first-1,500-of-20,872 archive prefix for disk budget) — applied before any content inspection but *not* pre-registered, and for EUSES the prefix collapses to essentially one category (791 of 796 files from the `database` folder, 9 from `cs101`, out of 4,652 files in 11 categories). EUSES-leg numbers should therefore be read as the *database category*, not the full corpus, and cross-corpus generality claims are correspondingly tempered; plus one **real production dbt project** (Mattermost's data warehouse, 254 models). The dbt leg's provenance is honestly *weaker* than the xlsx legs': the pre-registered GitLab target went private, the pre-registered fallback (a demo project) was bypassed for

the better real-production substitute, and the substitution rationale, leg harness, one harness fix (an identical source-oracle sentinel on both sides — a rename does not touch the warehouse), and results all attest in a single commit rather than a pre-inspection commit. Rules: run once; measurement-harness bugs fixable with disclosure (four were: the dbt sentinel above, a cell-association regex artifact, XML entity decoding, a per-file watchdog — after each xlsx-side fix the development-tier results re-verified byte-identical, and the two mismatch-reducing fixes are further justified by XML semantics independent of results: a self-closing `<c/>` cannot contain an `<f>` child, and `&quot;` denotes the same formula as a literal quote); the systems under test frozen.

Results against the pre-registered predictions (all scored in the repository's prediction ledger — 4 confirm, 3 disconfirm, 2 partial):

- **The central claim held on data development never touched: zero false certifications** across all 518 completed `certify(openpyxl edit)` calls — 503 refusals (158 EUSES + 345 Enron) plus 15 fail-closed errors — and 7 further edits failed before certification; **no would-corrupt edit was certified**.
- **Reference-shift arithmetic: 0 errors in 283,960 real formula cells** across the four ops (Enron: 170,796 cells, 100%). The checked-cell denominator is bounded by the truth grammar: cells outside it are skipped, never guessed — and the largest skipped class is *cross-sheet formulas* (on Enron, skipped cells exceed checked cells per op), which is exactly where the post-review qualifier defect below lives. The *only* defect the test found — 38 cells (0.034% of the 113,164 EUSES-leg cells), all in one Japanese workbook — was **not** a shift error: xlq double-encoded non-ASCII string literals (a Latin-1 byte misread in the rewrite path), silently corrupting literal *text* while shifting references correctly. This landed **exactly in the layer §3 explicitly labels trusted-not-proven** (the byte→token surface) — the scoping was load-bearing, not decorative. No foreign edit in the corpus was falsely certified because of it; we note precisely that over-refusal is guaranteed only for *correctly-encoding* foreign tools — a foreign tool sharing the identical byte-misread would produce byte-identical corrupted literals and be certified, since the mojibake transformation is deterministic and tool-independent. It also means the tool's own gates passed over the 38 corrupted cells (both certify sides share the encoder). The defect was frozen during the test per protocol and is fixed with regression tests since (§6). Predictions of zero mismatches: *disconfirmed* on EUSES, confirmed on Enron — reported as scored.
- **Fail-closed cost, measured:** the full guarded pipeline (transform residual ∪ own-certify not certified) refuses 19.6% (EUSES) and 32.0% (Enron) of eligible files — dominated by the `unverified_reference_part` denylist and loader limits on converted exotica. This is the honest price of soundness on real artifacts, and we report it as such rather than tuning it away post-hoc.
- **Would-corrupt prevalence:** 69.3% (EUSES, below our predicted range) and 89.2% (Enron, within it) of eligible files carry at least one reference a naive edit path would silently corrupt.
- **dbt (production project):** mini-adapter parse coverage 40.2% (*above* the predicted <30% — a favorable disconfirmation; 152 models with unmodeled Jinja fail closed), and on the 79-model closed subgraph the certify legs behaved ex-

actly as the theory demands: a faithful rename of a staging model with four real dependents CERTIFIED with no SQL executed; a dangling-reference botch REFUSED.

The test converts the fixture-tier headline into a test-tier statement: *the guard's soundness transferred to in-the-wild artifacts unchanged; its costs are real and now quantified; and its one failure was found by our own protocol, in the layer our own theory declared unproven.*

6. On finding our own defects

Successive adversarial reviews of these artifacts each landed a real hole: the tokenizer's syntactic proxies (silent corruption); the circular live-agent experiment (a tautological "0 false certifications"); the unimplemented defined-name *aliasing* boundary; and — most consequentially — a **reachable false certification** in the production certifier itself, where a foreign edit that shifts every cell formula correctly but leaves a defined name's target unshifted was certified with all-zero diffs, because the cell diff never compared non-cell references. We built the exploit, confirmed it, and closed it (defined names now compared against the tool's transform; data-validation, conditional-formatting, charts, pivots, and external links fail-closed). A subsequent review then falsified a *universal* phrasing of that fix by the same pattern — a foreign edit that reverts a `mergeCell`/`hyperlink`/`autoFilter` reference, which the transform shifts but the cell diff never compared, was certified — so we extended the net to those semantic structural references and reframed the boundary as an explicitly *enumerated denylist* whose completeness we do not claim to have proven. The coincidence-bound study (§5.8) surfaced label noise in our own headline A/B: two "corrupted" files carry zero references of any kind, so their flagged divergence is engine disagreement, not corruption — we corrected the headline from 86.6% to 85.5% confirmed-genuine and record the mislabel here. Most recently, the locked in-the-wild test (§5.9) found a **real silent-corruption defect in the tool itself**: the formula rewrite walked bytes and copied non-ASCII scalars as Latin-1 codepoints, double-encoding string literals — references shifted correctly, literal text corrupted, on exactly the trusted byte→token surface §3 scopes out of the proofs. Per the pre-registered protocol the tool stayed frozen for the test (the 38 affected cells are reported as measured); the defect is since fixed with regression tests covering the exact failing shape and all UTF-8 planes — and we state plainly that the tool's own certify gate passed over those 38 corrupted cells (both sides of an own-transform comparison share the encoder), so the defect was caught by the locked test's independent truth instrument, not by the guard. A granted post-test review round then found a **live sibling** by attacking the fix's boundary: the tokenizer's unquoted-sheet-qualifier grammar is ASCII-only, so a CJK-named sheet's unquoted qualifier (□□`01!CI3`) mis-tokenizes and the shift silently leaves the reference stale — a class present thousands of times in the locked corpus yet structurally invisible to the locked harness, whose truth grammar skips all cross-sheet formulas. Rather than extend the grammar (new trusted surface), the fix is fail-closed: a detector refuses any edit whose formulas carry an unquoted non-ASCII qualifier, with regression tests and an end-to-end refusal check; the locked numbers stay as measured. We report each as a fixed defect. This is part of the contribution: a certify-

or-refuse claim is only credible if its authors have tried hardest to break it — the record of what broke, and what the fix was, is the evidence that the remaining boundary is real, and it is why we are conservative about calling the corroboration anything more than corroboration, and the denylist anything more than a denylist.

7. Scope and limitations

The **verified surface** the certifier certifies (rather than refuses) is: row/column insert/delete *and row-block move* (§5.2b), single-sheet, in-grid coordinates, cell formulas over single-cell and range-endpoint references, with no defined-name/cell collision and (for move) no move-straddling range. On top of the sheet-cell diff, certify explicitly compares the reference-bearing constructs the transform shifts that a cell diff cannot see — defined-name targets and mergeCell/hyperlink/autoFilter `refs` — refusing any that differ from the transform, and fail-closes on data-validation, conditional-formatting, sparklines, charts, pivot tables, and external links. Two honesty caveats, both learned from this artifact's own failures. First, this compare-and-fail-close net is an **enumerated denylist**, not a machine-checked allowlist: its completeness over value-affecting non-cell references is *asserted, not proven*, so an unenumerated value-bearing construct would be silently certified — the exact failure the defined-name case exhibited until a reviewer built the exploit. (We keep certify's compare surface a superset of the transform's *value-bearing* write surface — every reference-bearing construct except the deliberately-excluded view-state `dimension/selection/pane/brk` — which makes the enumeration mechanically checkable against `has_ref_attr`, but that superset relation is a code invariant, not a proof.) Second, the guarantee is over **computed values**: pure view-state the transform also shifts — the used-range `dimension`, `selection`, frozen `pane`, and page `brks` — is deliberately *not* compared, because it is non-value-bearing and foreign tools legitimately vary it; a certified file may differ from the transform there without affecting any value. Value edits are out of the exact tier by construction. The **trusted base** (narrowed by Theorem 3 but not eliminated) is: the byte→token parse; the correctness of the shift map `σ` itself (the Z3-proved arithmetic, not re-proved in the graph layer); the asymmetric six-case delete clamp and its `#REF!` outcomes; the multi-cell interior of ranges (modeled by endpoints); the constructs absent from the token type; and the completeness of the non-cell denylist. The corroboration is bounded accordingly: value-preservation is necessary-not-sufficient (though now confirmed across all four structural ops and by two independent engines that agree, ruling out a single-engine artifact, and still not Excel); the confusion matrix's `FN=0` is by-construction over deviations-from-transform, so the informative test is the non-cell corruptor of §5.3, which found and closed a real false certification. The exact tier certifies a measured 37.5% of operations and 27% of whole tasks on a realistic edit-distribution study; the majority of real tasks are mixed and need the probabilistic tier — but by Theorem 2 a mixed task factors into a *certified structural scaffold* plus a bounded value-fill cone, and we measure this (`composition_coverage.py`): on mixed edits the scaffold is always certified and the audit surface collapses to the value-fill cone (100/84/68/52% of the artifact certified untouched as fills grow, mean 76%), so the certifiable *component* rises from **27% fully certified to 87% with a certified scaffold**. A verified byte-level tokenizer, and a model faithful to the clamp/`#REF!` algebra, are the natural

next steps.

8. Related work

The 2026 wave of LLM-spreadsheet-agent benchmarks documents precisely our failure class — Spreadsheet-RL [arXiv:2605.22642] names index-shift and reference-translation hazards and enforces an inspect–modify–verify loop; MBABench [arXiv:2605.22664] and BlueFin [arXiv:2605.30907] score structure and formula transparency — but all three verify **engine-in-the-loop** (recalculate and read) or by rubric/LLM judge, an evaluation MBABench's own authors call "inherently ambiguous, difficult to verify" (BlueFin similarly concedes theirs is "difficult to verify programmatically"); none certifies offline. Spreadsheet analysis and smell detection, structural-edit tools that reshape files without a fidelity property, and record/replay or unit-test approaches to spreadsheet correctness all differ from this work in the same way: they establish behavior on tested inputs, whereas we prove value-fidelity of the structural fragment for all inputs and all semantics, and gate untrusted edits on that proof. **Translation validation** [Pnueli, Siegel, Singerman, TACAS 1998] and verified compilation are the closest methodological analogues, and our production mode (§4.ii) is honestly an instance of TV's weakest form — accept iff the output equals a proven reference transform. The delta of this work over TV is twofold. First, the *direct-premise mode* (§4.i) does what TV does not: it decides the invariance theorem's hypothesis directly on the untrusted artifact, so its accept class is the entire *proven-certifiable* relabeling class (any producer's faithful relabeling, not only outputs identical to a canonical transform; whether certifiability extends past that class is stated open), and its soundness quantifies over **all** engines — TV assumes the semantics it validates against, whereas our setting's defining engine is opaque and absent. Second, TV has no analogue of our impossibility side: Theorem 5 shows the refusal half of certify-or-refuse is forced, bracketing the boundary rather than picking a point on it. Proof-carrying code shares the shape of "checkable evidence, small checker," and our decision procedure plays the checker's role with the certificate being the edit descriptor σ itself.

9. Conclusion

Verifiability, not capability, is the durable constraint on agent edits to opaque-semantics artifacts. We bracketed — machine-checked on both sides, middle ground stated open — what can be certified about such edits without the engine: the relabeling class is certifiable, witnessed by an executable decision procedure proven sound under every possible engine; anything introducing unwitnessed semantics is not, so certify-or-refuse is the only sound shape. The theory is load-bearing in the running system — the deployed checker agrees with the Lean decider, the production gate covers five structural operations on real spreadsheets, and the same core certifies dbt refactors engine-free — and every measurement is reported with its confound. The result is an honest, genuinely-verified boundary over an explicitly-scoped surface: the boundary a capable agent should be *required* to pass, not a patch for a weak one — and one that does not decay as agents improve, because it is grounded in the

artifact, not the model.